Pytorch

(Traditional ML vs ANN)

AGENDA

- convert Data to Tensor
- Build Pytorch ANN
- Build Early Stopping Class
- Build Optimizer, Loss & Early Stopping
- Train & Test ANN
- Comperson
- Conclusion

CONVERT DATA TO TENSOR

- **Purpose:** convert pandas/numpy arrays into torch.Tensor, the fundamental data structure for PyTorch
- **dtype:** torch.float32 ensures compatibility with network weights (which default to float32).
- **reshape(-1,1):** guarantees the targets are column vectors (N,1) matching the final layer output.

```
x_train_t=torch.tensor(x_train,dtype=torch.
float32)

x_test_t=torch.tensor(x_test,dtype=torch.float32)

y_train_t=torch.tensor(y_train.values,dtype=torch
.float32).reshape(-1, 1)

y_test_t=torch.tensor(y_test.values,dtype=torch.
float32).reshape(-1, 1)
```

BUILD PYTORCH ANN

- **Input layer:** nn.Linear(x_train.shape[1],512) connects the ~250-feature input to 512 hidden units.
- **Hidden layers:** progressively shrink dimensions 512 > 256 > 128 > 64 > 32 to capture non-linear feature interactions while controlling model size.
- **BatchNorm1d:** normalizes activations across the batch, stabilizes & speeds training, allows larger learning rates.
- **Dropout:** 0.2–0.4 to reduce overfitting, applied after activations.
- **Activation:** ReLU (Rectified Linear Unit) introduces non-linearity and avoids vanishing gradients.

```
class NN_model(nn.Module):
  def __init__(self):
     super(NN_model,self).__init__()
     self.model=nn.Sequential(
        nn.Linear(x_train.shape[1],512),
        nn.BatchNorm1d(512),
        nn.ReLU(),
        nn.Dropout(0.2),
        nn.Linear(512,256),
        nn.ReLU(),
        nn.Linear(256,128),
        nn.ReLU(),
        nn.Linear(128,64),
        nn.ReLU(),
        nn.Linear(64,32),
        nn.ReLU(),
        nn.Linear(32,1)
      )
  def forward(self,x):
    return self.model(x)
```

BUILD EARLY STOPPING CLASS

- **Early stopping:** a regularization technique used in iterative training (like neural networks) to halt training once the model stops improving on a validation set.
- **Patience:** If the metric doesn't improve for a set number of epochs (say, 20), stop training.
- **Best Weights:** Restore the model parameters from the epoch that achieved the best metric.

```python
class EarlyStopping:
    def __init__(self, patience=5,verbose=False, delta=0, path='checkpoint.pt'):
        self.patience = patience
        self.delta = delta
        self.best_score = None
        self.early_stop = False
        self.verbose = verbose
        self.counter = 0
        self.best_model_state = None
        self.path = path
        self.restore_best_weights = True

    def __call__(self, val_loss, model):
        score = -val_loss

        if self.best_score is None:
            self.best_score = score
            self.best_model_state = model.state_dict()
        elif score < self.best_score + self.delta:
            self.counter += 1
            if self.counter >= self.patience:
                self.early_stop = True
        else:
            self.best_score = score
            self.save_checkpoint(val_loss, model)
            self.counter = 0

    def save_checkpoint(self, val_loss, model):
        if self.verbose:
            print(f'Validation loss decreased ({self.val_loss_min:.6f} --> {val_loss:.6f}
).  Saving model ...')
        torch.save(model.state_dict(), self.path)
        self.val_loss_min = val_loss
```

OPTIMIZE & LOSS

- **Loss:** Mean Squared Error (MSE) measures average squared difference between predictions and true values standard for regression.

```
loss_F=nn.MSELoss()
optimizer=optim.Adam(model.parameters(),lr=0.001)
early_stopping = EarlyStopping(patience=20, delta=
0.01)
```

- **Optimizer:** Adam combines adaptive learning rates with momentum, generally faster and more stable than plain SGD for tabular data.

- **Early stopping**: Prevents overfitting training longer might reduce training loss but hurt generalisation and Saves time and compute.

TRAIN ANN

1-model.train(): sets layers like dropout/batchnorm (if present) to training mode.

2- Forward pass: compute predictions y_pred.

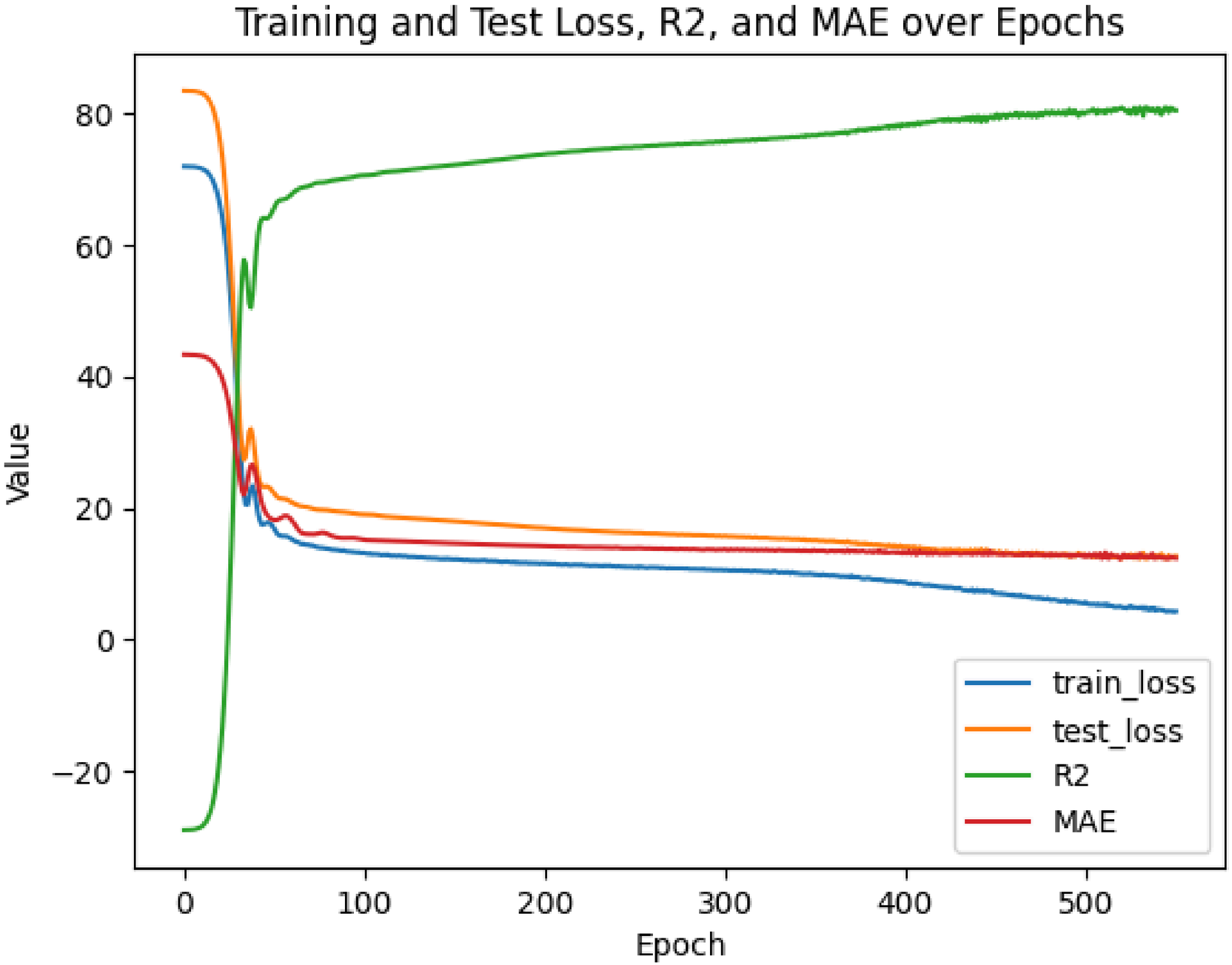
3- Loss: compare predictions to true targets.

```
epochs=2000
for epoch in range(epochs):
  model.train()
  y_pred=model(x_train_t)
  loss=loss_F(y_pred,y_train_t)
  optimizer.zero_grad()
  loss.backward()
  optimizer.step()
```

4- Zero gradients: zero_grad() clears old gradients to avoid accumulation.

5- Backward pass: loss.backward() computes gradients via automatic differentiation.

6- Update: optimizer.step() adjusts weights.

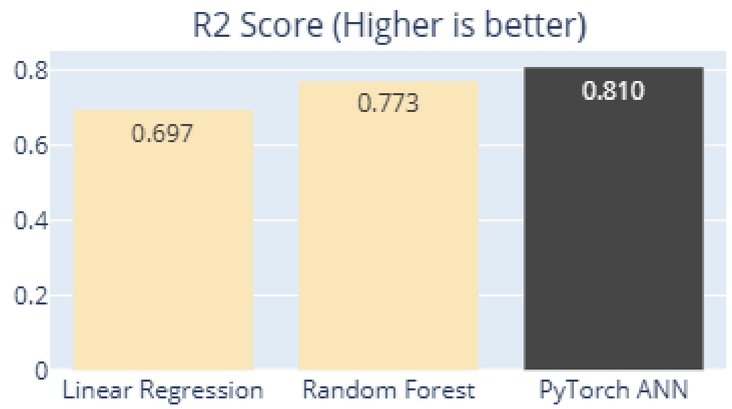

Training and Test Loss, R2, and MAE over Epochs
Value
Epoch
80
60
40
20
0
−20
0
100
200
300
400
500
train_loss
test_loss
R2
MAE

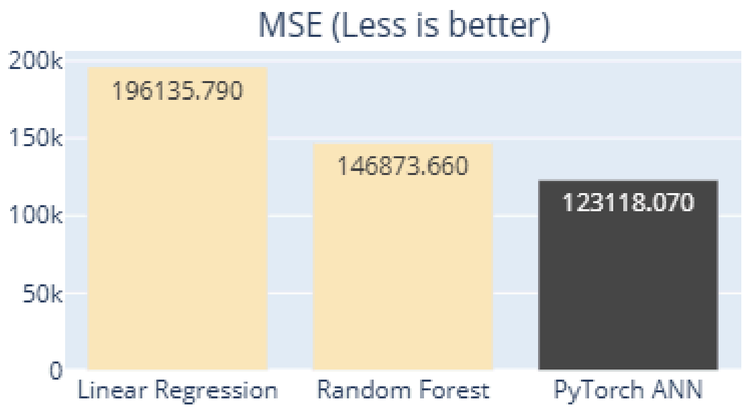
TEST ANN

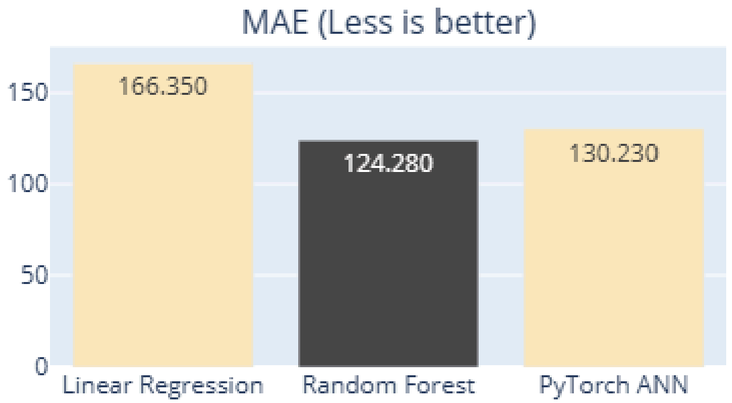
- **model.eval():** disables dropout/batchnorm randomness and uses running stats.
- **torch.no_grad():** stops gradient tracking to save memory/compute.
- **MSE/MAE:** from sklearn for easy reporting.
- **R2:** quantifies explained variance (1.0 = perfect fit, 0 = predicts mean).
- Printing every 10 epochs reveals both training loss and test metrics, letting you track learning and detect over/underfitting.

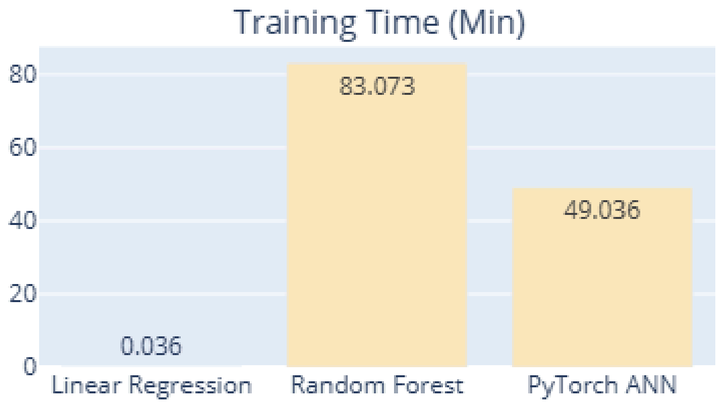
```
  model.eval()
  with torch.no_grad():
    y_pred_test=model(x_test_t)
    test_loss=loss_F(y_pred_test,y_test_t)
    val_loss_train.append(loss.item())
    val_loss_test.append(test_loss.item())
    R2_scores.append(r2_score(y_test_t,
y_pred_test))
    MAE_scores.append(mean_absolute_error(
y_test_t,y_pred_test))
    MSE=mean_squared_error(y_test_t,y_pred_test)
    MAE=mean_absolute_error(y_test_t,y_pred_test)
    R2=r2_score(y_test_t,y_pred_test)
    if epoch%10==0:
      print(f'Epoch {epoch+1}/{epochs}, Loss: {
loss.item():.4f}, Test Loss: {test_loss.item()
:.4f}, MSE: {MSE:.4f}, MAE: {MAE:.4f}, R2: {R2
:.4f}')

    early_stopping(test_loss, model)
    if early_stopping.early_stop:
        print("Early stopping triggered")
        break
model.load_state_dict(torch.load('checkpoint.pt'
))
```

Comparing models across the four metrics









CONCLUSION

- Traditional machine-learning models usually perform best on **structured tabular data** and often train faster with less complexity
- In this project, however, the **PyTorch ANN slightly outperformed the Random Forest**, helped by the large number of features, nonlinear patterns, and careful tuning of the network.
- Still, the real strength of neural networks appears with **unstructured data** such as images, audio, or text,This result shows that deep learning **can match or exceed classical ML in some tabular cases**, while its greatest advantage will emerge in future projects on unstructured data.

THANK YOU